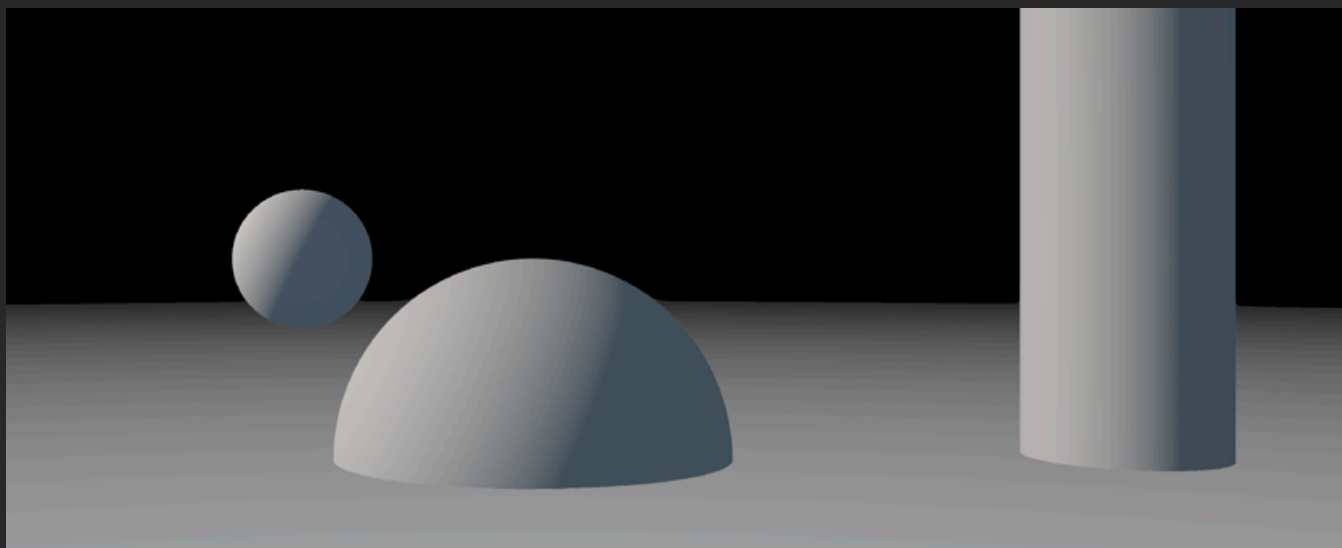


Inigo Quilez :: articles :: directional derivative - 2013

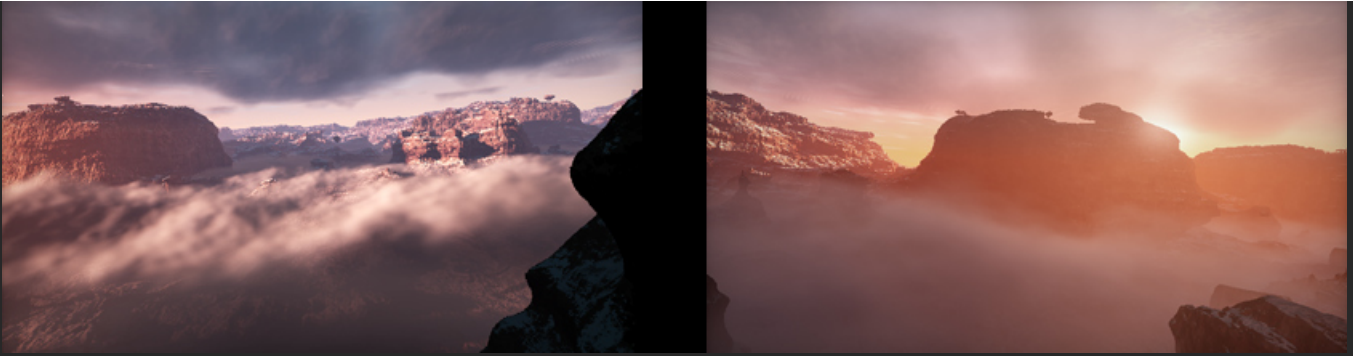


Intro

When we learn the concept of gradient at school it is presented to us as the primary way to describe and deal with directions and orientations, probably because it describes the local change of a function in a very general way. And so, a related concept called the "directional derivatives" is barely talked about, except perhaps as an intermediate concept to arrive at that of a gradient. And while the generality of the gradient probably grants the almost exclusive focus we give it, we shouldn't forget that sometimes we don't want the general solution or implementation, but the specialized and optimized one. Like when you want to compute lighting from a single light source for 3D objects efficiently without computing normals, or when you are painting volumetric clouds.



Diffuse lighting without surface normals (code: <https://www.shadertoy.com/view/XI23Wy>)



Fast (realtime) dynamic lighting on volumetric clouds with directional derivatives

Say you are doing some real-time rendering of volumetric clouds, and that you need to do some lighting and shaping without scattering and self shadowing computations. You need to go cheap, but even extracting the gradient or normal from the cloud volume that you need to do your regular Lambertian lighting is already expensive. You are probably evaluating your gradient by taking 4 or 6 samples of the volume, depending your implementation, only to then dot it with the light direction. Which works, but is very slow because evaluating your (possibly procedural) volumetric field 4 or 6 times is your bottleneck.

The idea

So now forget what your teacher told you about gradients and have a look to [this article](#) on the directional derivatives in the Wikipedia. In particular, look at this formula:

$$\nabla_{\mathbf{v}} f(\mathbf{x}) = \nabla f(\mathbf{x}) \cdot \frac{\mathbf{v}}{|\mathbf{v}|}$$

Now, if $\mathbf{x}$ was the point in space we are shading/lighting, and f was our SDF or cloud density field, then $f(\mathbf{x})$ would be the density at that point we are shading, and $\nabla f(\mathbf{x})$ the gradient (or 'normal'). At the same time, if $\mathbf{v}$ was the light direction, then the right side of the equation $\nabla f(\mathbf{x}) \cdot \mathbf{v}/|\mathbf{v}|$ would be nothing but our regular $\mathbf{N} \cdot \mathbf{L}$ Lambertian lighting... which according to the equation is equal to the directional derivative of the field taken in the direction of the light (left side of the equation)!

So basically, instead of extracting a general derivative in all possible directions and dot with the one direction of interest, you can measure the change (derivative) directly in that direction of interest. Or in other words, rather than taking 4 or 6 samples to extract a generic derivative or gradient, and then dot it with the light direction to do our lighting, we could simply sample the field no more than 2 times, at the current point and at a point a small distance away in the direction of the light (and divide by that distance of course). So, something

that is 4 or 6 evaluations can be reduced to one. Since one evaluations has been already done for computing the opacity of the volume, we are now really doing two evaluations rather than 5 or 7. Which is a massive speedup.

The code

So, let's say we have an SDF or a volumetric function called **map()**. On the left you can see the traditional way of doing your lighting based on gradients. To the right you can see the new way of performing lighting:

```
// map : SDF or density function
// eps: differential unit, base on req
vec3 calcNormal( in vec3 x, in float e
{
    vec2 e = vec2( eps, 0.0 );
    return normalize(vec3(map(x+e.xyy)-m
                          map(x+e.yxy)-m
                          map(x+e.yyx)-m
    )
}

void render( void )
{
    // ...
    float den = map( pos );
    vec3 nor = calcNormal( pos, eps );
    float dif = clamp(dot(nor,light),0.0
    // ...
}
```

```
// map : SDF or density function
// eps: differential unit, based on re
void render( void )
{
    // ...
    float den = map( pos );
    float dif = clamp((map(pos+eps*light
    // ...
}
```

If this code is called hundreds or thousands of times during a raymarch process because it's core to the volumetric raymarching process, then the gains can be massive, since the traditional method requires 2 evaluations per point while the new method only involves 7 evaluations per point. The code not only is 3.5 times faster, but also smaller, which is great if you are doing some size-coding based demo or shader.

Of course, the drawback is that this is only an advantage for a small number of light sources. So computing the normal might be advantageous anyways after 3 or 4 light sources, which is the most likely scenario (for example, to lit clouds you will want at least three light sources: the sun, the sky dome and the bounce coming from the ground).

Here are some pictures that show the new directional derivative based lighting versus the gradient based, and

also to no lighting at all, for comparison.

No lighting:



Directional derivative based lighting (3.5 times faster):



Traditional gradient based lighting:



If you are trying to see the differences between the directional derivative and the gradient method but don't spot them, that's because there are no differences.

It's also worth mentioning that the technique works also for solid surfaces of course, as illustrated in the first picture in this article. Here below I leave another example of the technique used with a solid SDFs, showing how to use this technique to get some lighting in a few bytes of code - click in the title to access the source code:

inigo quilez - learning computer graphics since 1994